Terminal-Bench 3.0 & 4.0

Verified manager guide — simplified, corrected and aligned with the current public TB3 rubric and current TB4 information

Verification date: 2 September 2026. This version corrects the previous guide's criterion-count/terminology issue and distinguishes official benchmark requirements from our simplified explanation.

1. Executive Summary

Terminal-Bench evaluates AI agents on difficult computer work performed through terminal/command-line environments. The agent must act inside an environment and leave a correct final state that can be checked.

Core outcome: resolution rate — the share of benchmark tasks an agent + model configuration successfully resolves.

TB3: the contribution goal was about 100 diverse tasks, with a target of at most about 30% solve rate from the best models at release. The released TB3 set used 74 task families. The central design principles were realistic paid work, clear instructions + robust verification, and genuine difficulty.

TB4: the current 4.0 release is a maintenance/quality update of the TB3 task set: 74 → 66 tasks, 8 removed, 20 revised, no new tasks in this cycle, resource calibration, and a flat 8-hour agent timeout.

Important: the three phrases 'real work + genuine difficulty + reliable verification' are the best management summary, not a replacement for the full implementation rubric.

2. Terminal-Bench in Plain English

Term	Simple meaning
Benchmark	A standardized test for comparing AI systems.
Agent	An AI system that reasons and takes actions using terminal/tools.
Task	One computer-work assignment the agent must complete.
Environment	The controlled/sandboxed computer environment in which the task starts.
Verifier	Automated checks that decide whether the final result is correct.
Resolution rate	Percentage/share of attempted benchmark tasks successfully resolved.
Trajectory	Recorded sequence of actions during an agent run.
Oracle/reference solution	Trusted solution used to demonstrate that the task is solvable and to support validation.

3. What Researchers Want From a Task

Core question	Meaning
Is it real work?	Could a professional plausibly be paid to do it?
Is it genuinely difficult?	Does it require meaningful professional expertise, reasoning, debugging or multi-step problem solving?
Can it be verified reliably?	Can an automated verifier determine correctness consistently, without depending on the particular method used?

One-line principle: difficult for the right reasons, professionally useful, and objectively checkable.

4. TB3 Task Acceptance Matrix — All 28 Current Rubric Criteria

The current public TB3 implementation-review automation says its implementation rubric evaluates **28 criteria**. The previous version of this guide incorrectly described the count as 27 and used a non-current 'well specified' label; this version replaces that

with the current **task_security** criterion and keeps the complete current list. ■cite■turn2search3■turn4view0■

#	Official criterion	Simple manager question	PASS looks like	FAIL / warning
1	verifiable	Can the verifier reliably decide pass/fail?	A working solution passes; verification is programmatic, reliable and repeatable. LLM-as-judge is exceptional and needs justification.	Subjective/flaky/non-deterministic verification.
2	solvable	Can a working reference solution be produced in a few hours by an expert who knows the idea?	Reference solution passes all tests and does not require unreasonable implementation work.	No working solution or task is impractical to implement.
3	difficult	Would an experienced professional find this genuinely challenging?	Difficulty comes from expertise/reasoning, not obscure facts or arbitrary work.	Average undergraduate can solve it in a few days, or difficulty is mostly tedium/tricks.
4	interesting	Is there a realistic professional use case?	Useful computer work, even if it serves a niche professional audience.	Toy puzzle, gimmick or artificial exercise with no real-world analogue.
5	outcome_verified	Can different valid approaches succeed?	Instructions state the desired end result and tests check the result.	Tests require a particular editor/tool/process without a legitimate reason.
6	anti_cheat_robustness	Can the agent bypass the intended work?	No leaked answers or trivial ways to fake success; account for root access and accessible environment.	Answer is exposed in the agent environment or verifier can be trivially bypassed.
7	task_security	Is the task safe for agents, reviewers and infrastructure?	No credential exfiltration, malicious downloads, host escape, destructive attacks, obfuscated payloads, prompt injection or unjustified network activity.	Any such behavior is present or unjustified.
8	functional_verification	Do tests execute/check behavior or outputs?	Behavior/API/output/state checks.	Primarily grep/string/keyword checks.
9	deterministic_reproducible	Would repeated runs behave consistently?	Dependencies pinned where appropriate; stable infrastructure; stochastic checks calibrated.	Unpinned relevant dependencies, mutable live service, or unreliable stochastic results.
10	essential_difficulty	Would failure indicate conceptual difficulty?	The hard part is technical reasoning/problem solving.	Most failures come from formatting, arbitrary precision or clerical details.
11	test_instruction_alignment	Do instructions and tests describe/check the same contract?	Every assertion maps to an instruction requirement and every requirement has coverage.	Hidden requirements or untested stated requirements.
12	novel	Can training-data memorization short-circuit the task?	Novel combination, custom environment/codebase or domain-specific scenario requiring exploration.	FizzBuzz, standard sorting, Sudoku solver or similar memorized exercise.

#	Official criterion	Simple manager question	PASS looks like	FAIL / warning
13	agentic	Must the agent explore, act, observe and adapt?	Environment exploration, debugging, file/process manipulation or multi-step interaction is essential.	One-shot generation, simple recall or a single command is enough.
14	reviewable	Can someone review it without being a domain expert?	Clear explanation/derivation for specialized ground truth.	Correctness depends on specialist knowledge with no explanation.
15	instruction_concision	Do instructions state what to achieve without giving a recipe?	Goal-first, concise, no unnecessary fluff/headings/step-by-step hints; use absolute paths.	Relative paths, solution hints, excessive procedure, roleplay/fluff, or tool lists the agent should discover.
16	solution_quality	Does the oracle actually solve the task?	Solution derives the result through real commands/code; large files are kept separately when appropriate.	It merely prints/copies the final answer.
17	environment_hygiene	Is the environment arranged correctly for evaluation?	Tests/solution are not baked into the agent image; test-only dependencies are handled appropriately; apt is managed correctly.	Tests/solution copied into runtime image or poor dependency/package hygiene.
18	structured_data_schema	Could an agent know the normative JSON/CSV/API/config schema?	Instruction/spec explicitly defines the schema; examples alone are insufficient.	Only examples are provided or schema is ambiguous.
19	typos	Could a naming error cause false failure?	Critical identifiers match across instruction, environment, solution and tests.	Typos create broken paths/commands/variables.
20	difficulty_explanation_quality	Does the explanation say why the task is fundamentally hard?	Specific reasoning/domain challenge, realistic data context when relevant, and real-world role/use.	Vague 'this is hard' explanation or explanation based only on model pass rates.
21	solution_explanation_quality	Can a non-specialist understand the approach?	Concise strategy consistent with actual solution files.	Missing, vague or contradictory explanation.
22	verification_explanation_quality	Can a non-specialist understand what the verifier checks and why?	Concrete checks, edge cases and justified tolerance/range bounds when used.	Vague/incomplete explanation or unjustified inequality/tolerance checks.
23	category_and_tags	Do category and tags reflect the real domain/skills?	Specific, relevant domain/skill tags; not generic labels such as hard/coding/task.	Wrong, generic or missing taxonomy.
24	task_name	Can a reader understand the task from its slug?	Lowercase kebab-case, specific, at most 5 hyphen-separated words.	Generic, vague, misleading, overly long or invalid name.
25	resource_configuration	Is the task hard for the agent rather than unnecessarily hard for the computer?	Resources and time are justified by actual needs and expert difficulty.	Unjustified excessive resources or timeouts too short/long.

#	Official criterion	Simple manager question	PASS looks like	FAIL / warning
26	task_readme	If present, does it add useful information?	Design notes, related tools/references or reviewer context not duplicated elsewhere.	Redundant or misleading README. N/A if absent.
27	expert_time_estimate	How long would a qualified expert take with the idea known?	Non-zero estimate consistent with task difficulty and timeout.	Missing/zero or clearly inconsistent estimate.
28	task_toml_schema	Are configuration fields legitimate Harbor/TB3 fields?	No invented/ignored fields; fields match the benchmark schema.	Extra/invented fields that are silently ignored or misleading.

5. The Three Core Principles

1 — Realistic: It represents useful computer work someone could plausibly be paid to perform.

2 — Genuinely difficult: The challenge comes from expertise, reasoning, debugging, multi-step work or domain complexity—not arbitrary formatting, obscure trivia or artificial tricks.

3 — Reliably verifiable: Tests establish that the intended outcome was achieved. Different legitimate ways of solving the task should generally be allowed.

These are a **manager-level summary**. The 28-criterion rubric is the detailed implementation review.

6. What a Proper TB3 Task Contains

Component	Purpose
instruction.md	States the goal/outcome for the agent.
environment/	Builds the starting environment, typically with a Dockerfile and task inputs.
solution/	Contains the trusted reference/oracle solution.
tests/	Contains verification logic that checks the completed task.
task.toml	Stores metadata, timeouts, resources and other supported configuration.
README.md (optional)	Provides useful development/reviewer context not intended as task instructions.

7. Common Reasons a Task Is Weak or Rejected

Problem	Why it weakens the benchmark
Simple textbook problem	Often tests memorization/basic coding rather than frontier agentic ability.
Toy puzzle / gimmick	Does not represent useful professional work.
Formatting is the main difficulty	Models can fail for clerical reasons rather than the intended capability.
Keyword/grep-only verifier	Easy to game and does not establish actual functionality.
Hidden test requirements	Creates an unfair mismatch between task instructions and grading.
Forced process/tool without reason	Measures compliance with a method instead of the outcome.
Mutable live service is task-critical	Can make outcomes non-reproducible.
Hardcoded reference answer	Does not demonstrate genuine solvability.
Unsafe code/network/credential behavior	Creates security and infrastructure risk.

8. Metrics — What to Measure and How to Read It

Metric/signal	Meaning	Use
Resolution rate	Share of tasks resolved successfully.	**Primary benchmark outcome.** Higher is better.
Uncertainty / confidence interval	Statistical uncertainty around a reported result.	Do not overinterpret tiny differences when uncertainty is substantial.
Task-level pass/fail	Whether an individual task was resolved.	Useful for finding capability-specific failure patterns.
Tokens	Tokens used during the evaluation.	Efficiency/cost signal, not the correctness metric.
Cost	Reported model/agent cost for an evaluation.	Useful for cost-effectiveness comparisons.
Trajectory/actions	Recorded agent behavior.	Useful for diagnosing failures and reward hacking.

Derived metric: your team may calculate cost-per-solved-task, but label it as a derived analysis rather than an official primary Terminal-Bench metric.

9. TB3 → TB4 — Exact High-Level Difference

Area	TB3	TB4	Meaning
Task set	74 tasks	66 tasks	8 tasks removed.
Revisions	Baseline	20 tasks revised	Quality/maintenance updates to existing tasks.
New tasks in this release	—	0	TB4 focused on maintenance rather than adding new tasks.
Removed tasks	—	8	2 saturation, 2 refusals, 2 public solutions, 2 quality/platform issues.
Agent timeout	Task-specific settings	Flat 8 hours	Reduces failures caused by insufficient time budget.
Resources	Task-specific	Recalibrated	CPU/memory/time were adjusted to reduce infrastructure noise.
Primary score	Resolution rate	Resolution rate	Core outcome remains task completion.
Comparability	Version-specific	Version-specific	Do not compare percentages as if task set/conditions were identical.

TB4 is therefore best understood as a **quality/maintenance release of the evolving benchmark**, not simply 'TB3 with a new number'. The task set and evaluation conditions changed. ■cite■turn5search0■turn5search8■

10. Why TB4 Removed Tasks

Reason	Simple explanation
Saturation	A task became too easy for frontier systems and stopped separating them meaningfully.
Refusals	Models refused the task for reasons that did not represent the intended capability.
Public solutions	Solutions became publicly available, weakening the task as a clean capability test.

Reason	Simple explanation
Quality/platform issues	Reliability, specification or compatibility problems made the task unsuitable without fixes.

The current TB4 reporting further specifies that saturation was judged using repeated runs across model classes/families; the benchmark team described saturated tasks as those no longer meaningfully differentiating systems. ■cite■turn5search4■turn5search0■

11. Current TB4 Leaderboard Snapshot — Context Only

The leaderboard reports complete **model + agent/harness configurations**, not isolated model capability. For example, 'GPT-5.6 Sol + Codex' is a system configuration. ■cite■turn5search0■

Rank	System	Resolution
1	Claude Opus 5 + Claude Code	51.8% ± 3.4
2	Claude Fable 5 + Claude Code	44.5% ± 3.8
3	GLM-5.3 + Claude Code	41.8% ± 3.2
4	GPT-5.6 Sol + Codex	37.3% ± 3.8
5	Claude Opus 4.8 + Claude Code	23.6% ± 3.6
6	GPT-5.6 Terra + Codex	21.5% ± 3.3
7	Grok 4.6 + Grok Build	20.3% ± 3.1
8	GPT-5.6 Luna + Codex	17.3% ± 2.8
9	Grok 4.5 + Grok Build	12.4% ± 2.6
10	Claude Sonnet 5 + Claude Code	12.4% ± 3.1

This is a current snapshot and can change. Tokens and cost shown on the current TB4 leaderboard are totals for the full evaluation, not per task, and 'effort' is a harness setting rather than a Terminal-Bench parameter. ■cite■turn5search0■

12. What Your Team Should Actually Do

Step	Action	Output
1	Understand TB3 purpose and contribution bar.	Benchmark summary.
2	Score the proposed task against all 28 criteria.	TB3 acceptance matrix.
3	For each criterion, record evidence and PASS/FAIL.	Review evidence.
4	Build/validate environment, solution and tests.	Working task package.
5	Run oracle/reference solution and repeated verification.	Reproducibility evidence.
6	Run agents and inspect failures/trajectories.	Agent failure analysis.
7	Study TB4 changes and ask whether the task will remain useful as models improve.	TB4 comparison + longevity assessment.
8	Revise, reject or submit the task based on evidence.	Manager-ready decision.

13. Manager's Final PASS Checklist

1. Can the verifier reliably decide pass/fail?

2. Can a working reference solution be produced in a few hours by an expert who knows the idea?

3. Would an experienced professional find this genuinely challenging?

4. Is there a realistic professional use case?

5. Can different valid approaches succeed?

6. Can the agent bypass the intended work?

7. Is the task safe for agents, reviewers and infrastructure?

8. Do tests execute/check behavior or outputs?

9. Would repeated runs behave consistently?

11. Do instructions and tests describe/check the same contract?

13. Must the agent explore, act, observe and adapt?

16. Does the oracle actually solve the task?

23. Do category and tags reflect the real domain/skills?

27. How long would a qualified expert take with the idea known?

14. Final Takeaway

Terminal-Bench is a practical test of whether an AI agent can do difficult computer work end-to-end. The agent receives a controlled environment, must investigate and act through the terminal, and is judged on whether the final outcome passes reliable verification.

For TB3: our main research task is to understand the task-quality/acceptance bar. The 28-criterion matrix is the working checklist.

For TB4: our main research task is to understand continuous benchmark maintenance: removing saturated/compromised/problematic tasks, fixing existing tasks, recalibrating resources and timeouts, and re-running the leaderboard under the new conditions.

> **One sentence to tell your manager:** 'A strong Terminal-Bench task is realistic professional computer work that is genuinely difficult for the right reasons, requires real agentic interaction, and has a reliable verifier that proves the final result is correct.'

15. Sources and What They Support

Official TB3 contribution call: target of ~100 diverse tasks; ≤30% solve-rate target at release; real compensated work; clear instructions + robust verification; genuinely difficult. ■cite■turn0search0■

Current TB3 public implementation rubric: 28 named criteria, including task_security, task_name, expert_time_estimate and task_toml_schema. The exact current rubric is the authority for implementation review. ■cite■turn4view0■turn2search3■

Current TB3 task template: shows required metadata such as difficulty/solution/verification explanations, category/tags, expert_time_estimate_hours, plus verifier/agent/environment configuration. ■cite■turn2search6■

TB3 contribution structure: the public contributing guide documents the task package structure including environment, solution, tests, task.toml and optional README. ■cite■turn2search5■

Current TB4 overview: 66 tasks, down from 74; 8 removed; 20 revised; 0 added; resolution rate as the core score; current leaderboard methodology and model+agent interpretation. ■cite■turn5search0■

TB4 continuous-QA explanation: resource calibration, timeout increases, task fixes and removals were bundled into the 4.0 release. ■cite■turn5search8■

Important source-discipline note: this guide is a simplified working reference. If your team actually submits a task, the live official repository/rubric is the final authority because the benchmark can evolve. The guide should not be treated as a frozen copy of the repository.